University *of Ljubljana*
Faculty *of Computer and Information Science*

Natural language processing course

Angel Naumov, Naum Spasev, Marija Koshtrevska

Abstract

Large language models have difficulty providing accurate, up-to-date answers in rapidly evolving fields like competitive gaming. This project introduces a domain-specific AI assistant for League of Legends that uses Retrieval-Augmented Generation (RAG). The system indexes patch notes, champion statistics, and match history data and uses this information to retrieve relevant context at query time and generate informed, accurate responses. The assistant can answer questions about champion strengths and weaknesses and predict the viability of champion picks based on retrieved patch and matchup data. Evaluation combines automated retrieval metrics with human assessments of response quality and usefulness. The expected results will demonstrate that a domain-specific RAG reduces inaccurate responses and improves precision compared to a standalone LLM.

Keywords

Domain-Specific AI, Natural Language Processing, Retrieval-Augmented Generation, Game Analytics

Advisors: Slavko Žitnik

Introduction

In recent years, large language models (LLMs) have demonstrated impressive abilities in understanding and generating human-like text across various fields. However, despite their versatility, these models struggle with tasks requiring the latest specialized knowledge or a deep understanding of context.

These limitations are noticeable in competitive gaming environments, such as League of Legends. This complex multiplayer game has a continuously shifting strategic landscape due to frequent patch updates. The choices players make are based on a complex interaction of various factors, including the strengths and weaknesses of the champion, the items chosen, and past performance trends. This is something that a model based on statistics struggles to keep up with.

This work addresses these challenges by developing a domain-specific conversational assistant based on a RAG framework. Rather than relying on knowledge encoded during pretraining, the system dynamically retrieves relevant information from a tailored knowledge base containing patch notes, champion statistics, and match history data. This context is then incorporated directly into response generation. This design is particularly well-suited to League of Legends: when a new patch is released, only the knowledge base requires updating, not the model itself. The result is a system that can accurately answer nuanced questions about champion performance, recent balance changes, and matchup recommendations, surpassing the capabilities of standalone LLMs.

Related Work

Earlier machine learning studies in League of Legends have mostly concentrated on match outcome prediction and player performance assessment, attaining robust outcomes through gradient boosting, tensor factorization, and event-driven win probability models [1, 2, 3]. Jiang et al. [4] notably demonstrated that incorporating patch version context directly into predictive models improves accuracy. This confirms that game knowledge is inherently version-dependent and rapidly evolving.

However, these models primarily operate on structured data and lack mechanisms for interactive knowledge access. This has led to growing interest in NLP approaches.

Lee et al. [5] introduced a voice-based large language model companion for beginner League of Legends players in their 2026 study, LeagueBot, reporting significant reductions in cognitive challenge and perceived tension in a controlled experiment with 33 players. However, participants reported receiving recommendations for nonexistent items. The authors attributed this hallucination problem to the LLM's lack of access to current game data. RAG was explicitly proposed as a solution, but latency concerns in fast-paced gameplay prevented its implementation. Motivated by these limitations, this work presents an RAG-based assistant for League of Legends that retrieves current patch notes, champion statistics, and match data, making it possible to provide practical, up-to-date responses.

Data

We leverage the official Riot API[6] alongside player data sourced from OP.GG[7] as our data. Beyond in-game performance metrics, we integrate official documentation and community knowledge to guide feature engineering and model training. Additionally, we utilize balance patch releases from Riot Games to capture and account for evolving gameplay mechanics across different patch versions.

Game Rules and Domain Knowledge

The foundational layer of the knowledge base consists of official and community-maintained documentation describing the rules, objectives, and mechanics of League of Legends. These sources provide structured, stable textual descriptions of core concepts such as champion roles, map objectives, itemisation, and win conditions. Since this content changes infrequently, it serves as a static background knowledge layer, ensuring consistent understanding of game fundamentals.

Beginner and Strategy Guides

To ensure the assistant can respond to practical, player oriented queries in natural language, beginner and strategy guides are incorporated. These documents reflect how real players formulate questions and what useful answers look like in practice. This category also informs the construction of the evaluation question set, as the phrasing and topics closely mirror the queries the assistant is expected to handle.

Patch Notes

Patch notes represent the primary and most critical component of the RAG knowledge base. Riot Games releases balance updates approximately every two weeks, introducing champion buffs, nerfs, item changes, and systemic adjustments that fundamentally alter the competitive landscape. Each patch document is chunked, embedded, and indexed into the vector store, allowing the retrieval pipeline to surface the most relevant balance changes in response to user queries. This design enables the knowledge base to be updated with each new patch without any modification to the underlying model.

Champion Statistics and Meta Data

Champion performance statistics are computed from match data collected via the Riot API, calculating win rates, pick and ban rates, optimal item builds on a per-patch and per-rank basis. This data is inherently dynamic. A champion considered strong in one patch may be significantly weaker following a targeted nerf, making it well-suited for RAG retrieval. When a user asks whether a specific champion is viable in the current meta, the retrieval pipeline surfaces the most recent computed statistics, grounding responses in empirical data derived from observed gameplay.

Match Data

Historical match data is obtained via the Riot Games Developer API, which provide granular per-game information including champion selections, item builds, kill events, gold progression, and final outcomes. This data serves primarily as ground truth for evaluation: champion pick recommendations can be assessed against actual match outcomes from Diamond+ tier games to quantify predictive accuracy.

Methods

Retrieval-Augmented Generation Pipeline

Motivation and Data Scope

A Retrieval-Augmented Generation (RAG) pipeline was implemented to allow the language model to answer questions about League of Legends patch changes accurately, without hallucinating statistics. The base model (`meta-llama/Meta-Llama-3-8B-Instruct`) has a knowledge cutoff and cannot reliably recall exact numerical changes from recently released patches.

Patch notes were collected for patches 26.1 through 26.9, covering the full 2026 Season 1 ranked period. This range was chosen because each new season in League of Legends brings major changes to champions, items, and overall game mechanics, making older information less relevant. By focusing only on the current season, the system is able to provide answers that reflect the current meta rather than outdated gameplay patterns.

In total, the scraping process produced **210 meaningful chunks** of information. Each chunk represents a specific change, such as a champion update or item adjustment, and includes metadata like patch version, date, section, subject, and type of change.

Pipeline Architecture

The pipeline was built using LangChain with the following components:

- **Embedding model:** `sentence-transformers/all-MiniLM-L6-v2` to encode chunks into dense vector representations.
- **Vector store:** FAISS index for efficient approximate nearest-neighbour search.
- **Retriever:** Maximum Marginal Relevance (MMR) with $k = 5$, fetch_k $= 20$, $\lambda = 0.6$ to balance retrieval relevance and diversity.
- **LLM:** Llama 3 8B Instruct loaded in 4-bit NF4 quantization via `bitsandbytes`.
- **Chain:** Retrieved chunks, formatted with patch version and section metadata, are injected into the system prompt alongside the user question.

A conversational variant was also implemented, where follow-up questions are first rewritten into standalone queries using the chat history before retrieval. This prevents vague follow-ups such as *"was that too hard?"* from retrieving irrelevant chunks.

Results

The system was evaluated qualitatively on five queries. Results are summarised in Table 1.

Query	Outcome
Kassadin nerfs	Correctly cited health growth nerf in patch 26.4: 119 → 113
Gwen nerfs	Correctly identified passive monster damage cap nerf in patch 26.2 with Riot's stated reasoning
Hubris item changes	Correctly traced changes across patches 26.5 and 26.9
Seraphine bug fixes	Correctly listed four specific bug fixes from patch 26.9
Anivia viability	Correctly refused to answer — champion not present in patch data

Table 1. Qualitative evaluation of the RAG pipeline across five test queries.

A key qualitative comparison was performed on Varus patch 26.2 changes. Without RAG, the model hallucinated plausible but entirely incorrect ability statistics. With RAG, it correctly cited the exact values.

The conversational test on Lillia across patches 26.2 and 26.5 demonstrated correct multi-patch reasoning, including Riot's stated rationale for reverting earlier nerfs. When asked about item build changes absent from the corpus, the system correctly responded: *"I don't have that information in the current patch data"*, confirming the system does not hallucinate beyond its context.

Fine-Tuning Data Pipeline

Motivation

While the RAG system retrieves patch note context at query time, it lacks grounding in actual gameplay data — how Diamond-level players build champions, perform, and win or lose in practice. To address this, we constructed a supervised fine-tuning (SFT) dataset from real ranked match data, following the QLoRA approach demonstrated in the course material.

Data Collection

Match data was collected from Diamond EUW ranked players using the Riot Games API. Player names and tags were scraped from op.gg, converted to PUUIDs via the Riot Account API, and used to fetch match histories. A total of 102 Diamond EUW players were collected, with 20 matches per player, yielding approximately 2,000 matches in total.

For each match, structured fields were extracted including champion, position, patch version, items, keystone rune, summoner spells, KDA, CS, gold, damage dealt, vision score, kill participation, turret kills, game duration, allied team, and enemy team. The resulting records were stored as `match_data.json`.

Synthetic Q&A Generation

Raw match statistics are not directly suitable for language model fine-tuning, which requires natural language instruction-response pairs. We therefore used `meta-llama/Meta-Llama-3-8B-Instruct` — the same model used in the RAG pipeline — to generate natural language answers from each match record.

For every match, four questions were generated covering: item build and runes, performance summary, win/loss analysis, and team composition. The model was prompted with a structured context block derived from the match fields and instructed to respond in 2–4 natural sentences using exact statistics from the data.

Each training example was stored in the format expected by `SFTTrainer`:

```
### Human: How did Zed perform in this
    Jungle game on patch 16.5?

### Context: Champion: Zed | Position:
    Jungle | Patch: 16.5 ...

### Assistant: Zed had a dominant
    performance, finishing 7/3/14 with
232 CS and 16,070 gold in under 30 minutes.
    His 66% kill
participation shows consistent map presence
    throughout the game.
```

Fine-Tuning

The base model was fine-tuned on the 4,876 instruction-response pairs using QLoRA, which freezes the original model weights and trains only a small set of low-rank adapter matrices. This keeps the model's general language capabilities intact while steering it toward Diamond-level gameplay knowledge — item choices, rune rationales, and performance patterns specific to high-ELO ranked play. Training ran for a single epoch, which was sufficient given that the dataset was synthetically generated with consistent formatting and low noise.

Qualitative Evaluation

The fine-tuned model was evaluated by posing eight natural language questions about Diamond EUW gameplay without providing any match context in the prompt. The responses were assessed qualitatively against known high-level play standards.

The model demonstrated clear adaptation to the target domain — answers were fluent, confidently phrased, and structured in the style of a gameplay analyst. Champion identities were correctly understood: Hecarim was described as a mobility-focused jungler and correctly assigned Flash and Ghost, Ahri's roaming potential was accurately highlighted, and Zed was correctly framed as an assassin.

However, several factual errors were present. The suggested item build for Zed included Mejai's Soulstealer and

Infinity Edge — neither of which are appropriate for the champion. Lulu support was described as building Liandry's Torment and Axiom Arc, whereas enchanter items such as Moonstone Renewer and Shurelya's Battlesong are standard. The model also incorrectly dismissed Hubris as a weak first item for assassins, despite it being a common and effective choice in Diamond play. Win rate figures cited for specific champions were stated with false precision and likely fabricated.

These errors are a direct consequence of the data pipeline: the training pairs were generated synthetically using the same base model, without any ground truth verification against actual item builds or champion statistics. The model therefore learned the *format and tone* of high-level analysis effectively, but inherited and reinforced factual inaccuracies present in the generated data. This is a known limitation of purely synthetic instruction tuning and motivates the hybrid RAG approach discussed in the future work section, where retrieved patch and build data would provide factual grounding at inference time.

Future Work

This project evaluated RAG and supervised fine-tuning as two separate approaches. A natural next step would be to combine them: using the fine-tuned model as the backbone of the RAG pipeline, so that the model brings domain-adapted language and reasoning while still retrieving up-to-date patch note context at query time.

References

[1] Ruben Ferreira and Jana Faganeli Pucer. Quantifying player death impact in league of legends. In *Proceedings of the 11th Student Computing Research Symposium*, pages 51–54, 2025.

[2] Philip Z Maymin. Smart kills and worthless deaths: esports analytics for league of legends. *Journal of Quantitative Analysis in Sports*, 17(1):11–27, 2021.

[3] Lincoln Magalhaes Costa, Rafael Gomes Mantovani, Francisco Carlos Monteiro Souza, and Geraldo Xexéo. Feature analysis to league of legends victory prediction on the picks and bans phase. In *2021 IEEE conference on games (CoG)*, pages 01–05. IEEE, 2021.

[4] Julie Jiang, Kristina Lerman, and Emilio Ferrara. Individualized context-aware tensor factorization for online games predictions. In *2020 International Conference on Data Mining Workshops (ICDMW)*, pages 292–299. IEEE, 2020.

[5] Jungmin Lee, Inhee Cho, and Youngjae Yoo. Leaguebot: A voice llm companion of cognitive and emotional support for novice players in competitive games. *arXiv preprint arXiv:2602.01213*, 2026.

[6] Riot Games. Riot games developer portal. Accessed: 2026-03-20.

[7] OP.GG. Op.gg. League of Legends Statistics and Analytics Platform.